Fondamenti di SQL Server: concorrenza, locking e isolamento

Gianluca Hotz

ghotz@ugiss.org

SQL START! 2025

Sponsors & Organizers

Bio

- Gianluca Hotz | @glhotz | ghotz@ugiss.org
- Consulente Indipendente
  - 30 anni con SQL Server (dalla versione 4.21 nel 1996)
  - «Database modeling & development, sizing & administration, modernization (upgrades & migrations), performance tuning, security»
- YNAP SQL Server DBA Team (part-time)
- «Community»
  - 25+ anni Microsoft MVP SQL Server/Data Platform (dal 1998)
  - «VMware Experts SQL Server»
  - Fondatore e presidente UGISS (ex «PASS Chapter»)

Proprietà ACID

- Atomicità
  - transazione eseguita in modo indivisibile (tutto o niente)
- Consistenza (o coerenza)
  - database deve essere coerente con i vincoli e non contraddittorio prima e dopo l'esecuzione della transazioni
- **Isolamento**
  - livello di interferenza tra transazioni in esecuzione contestuale
- Durabilità (o persistenza)
  - le modifiche confermate devono essere persistite in modo permanente (nessun guasto successivo potrà determinare la perdita dei cambiamenti)

https://en.wikipedia.org/wiki/ACID

Controllo delle transazioni

Controllo delle transazioni in T-SQL

- Transazioni esplicite
  - comando BEGIN TRANSACTION
  - comando COMMIT/ROLLBACK TRANSACTION
- Transazioni implicite
  - a livello di singolo comando
  - opzione SET IMPLICIT_TRANSACTIONS ON
- Controllo immediato
  - rollback dipende dalla criticità dell’errore

Transazioni nidificate

- Supportate solo sintatticamente
  - c’é al più UNA sola transazione aperta per sessione
  - commit delle transazioni interne ignorate

  - BEGIN TRANSACTION aumenta @@trancount
  - COMMIT TRANSACTION decrementa @@trancount
  - quando @@trancount raggiunge lo 0, COMMIT
  - ROLLBACK imposta @@trancount a 0

  - serve per il controllo delle transazioni delle stored procedure

Transazioni nidificate e punti di salvataggio

- Sono supportati punti di salvataggio
  - comando SAVE TRANSACTION
  - comando ROLLBACK al punto di salvataggio

Demo

- Controllo transazioni in T-SQL
- Transazioni nidificate in T-SQL

Approfondimenti

- http://www.sommarskog.se/error_handling/Part1.html
- http://www.sommarskog.se/error_handling/Part2.html

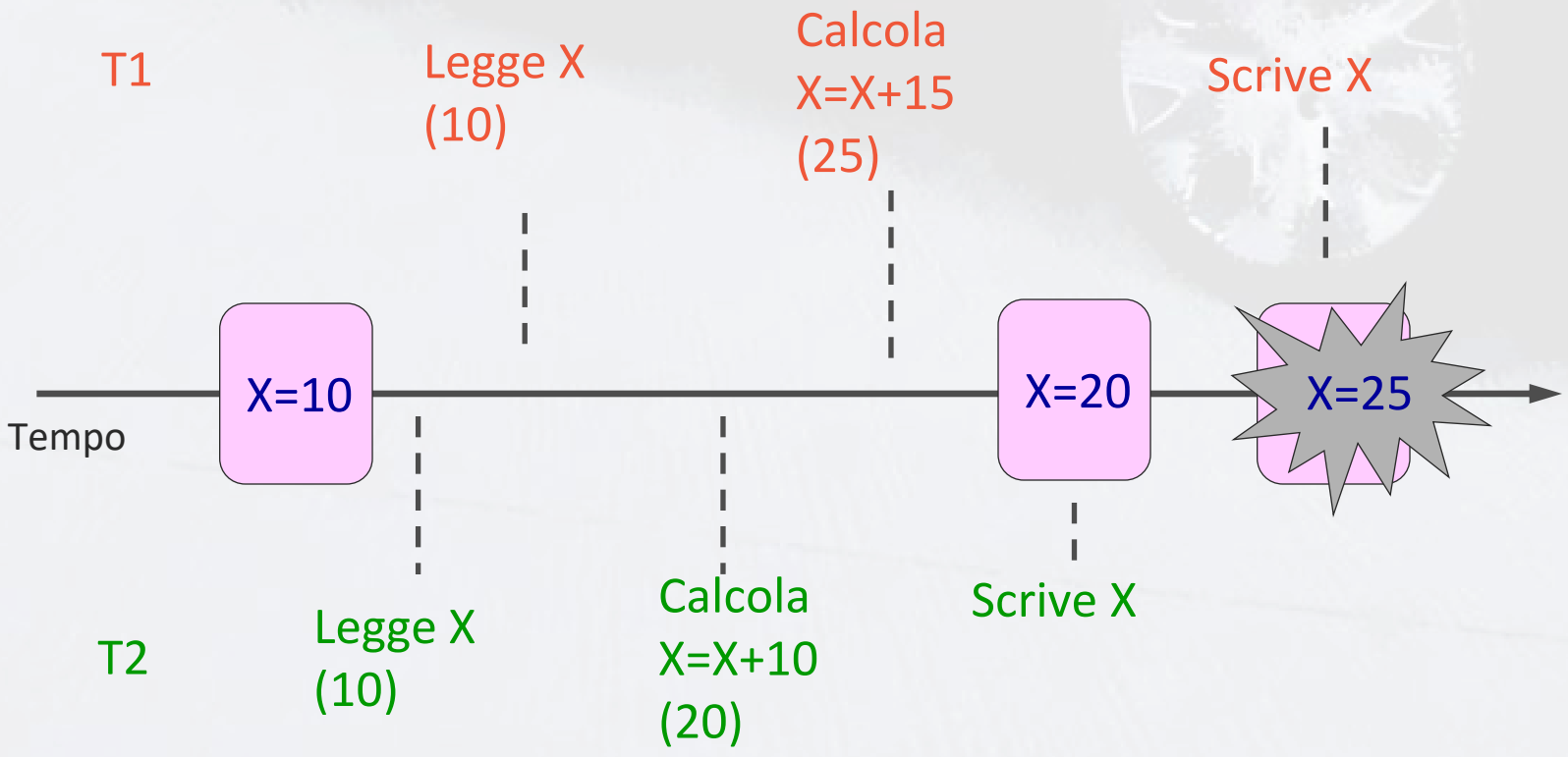
Isolamento delle transazioni

Isolamento delle transazioni

- Idealmente: serializzazione delle transazioni
  - massima consistenza
  - minor concorrenza
- Se le transazioni non sono coordinate
  - perdita di aggiornamenti (lost updates)
- Se transazioni sono parzialmente isolate
  - letture inconsistenti («dirty reads»)
  - letture non ripetibili («non-repeatable reads»)
  - letture fantasma («phantoms»)
  - conflitto in aggiornamento («update conflict»)

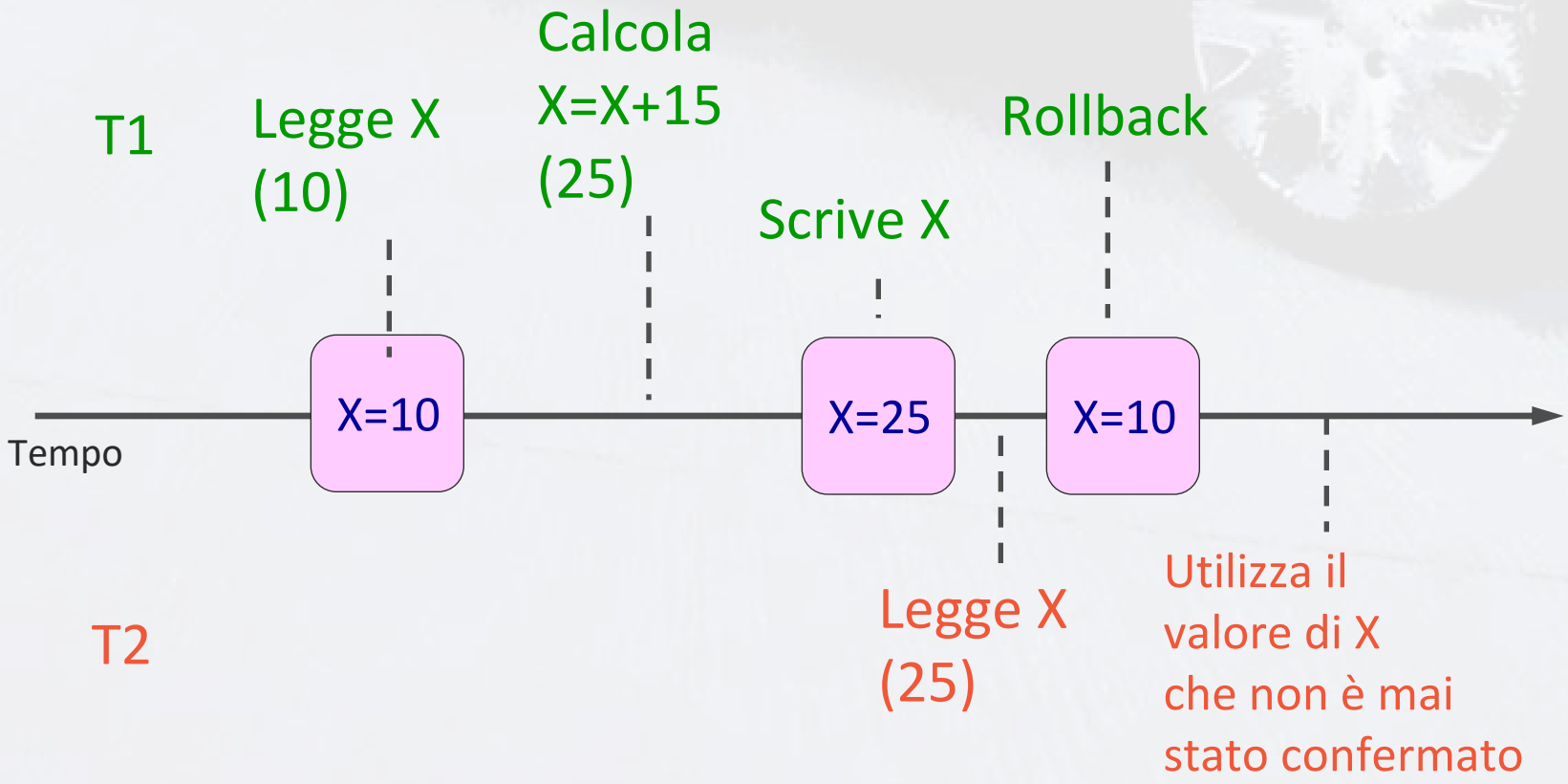
Perdita di aggiornamenti

- Aggiornamento non sincronizzato



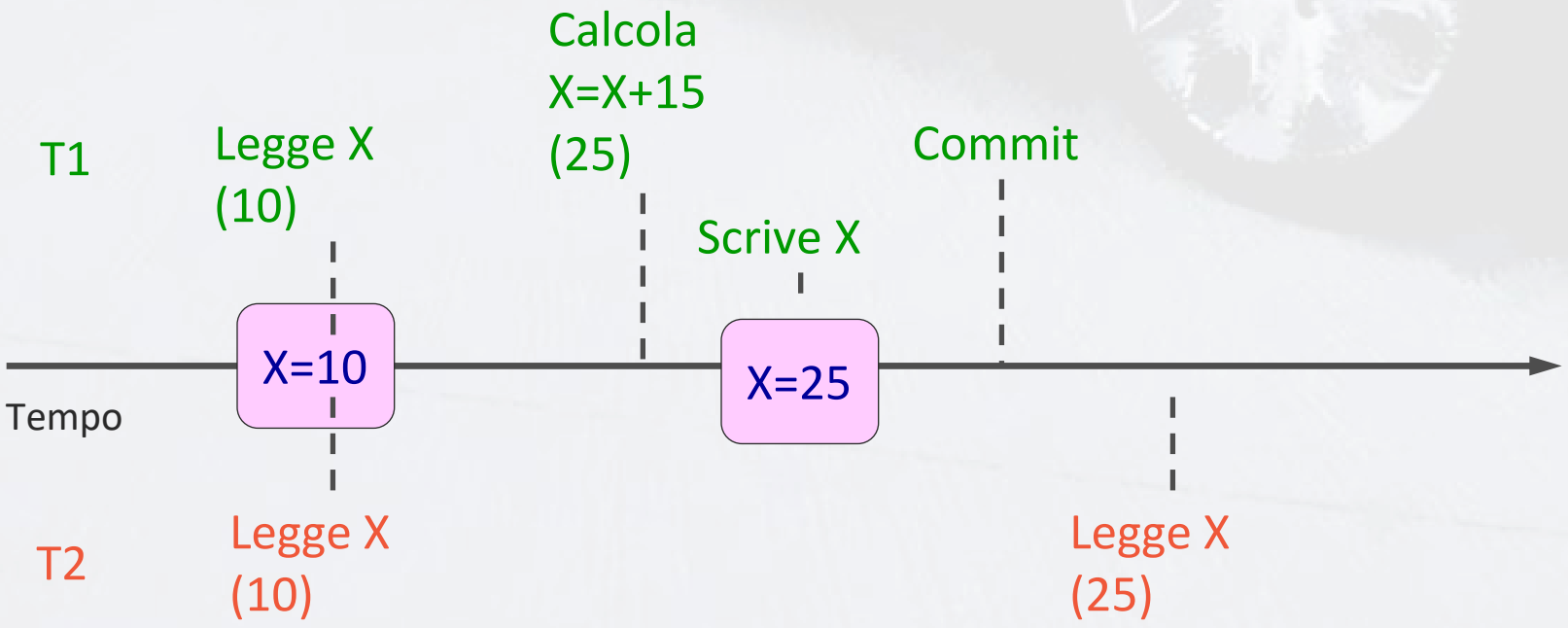
Letture inconsistenti

- T2 vede le modifiche non confermate di T1



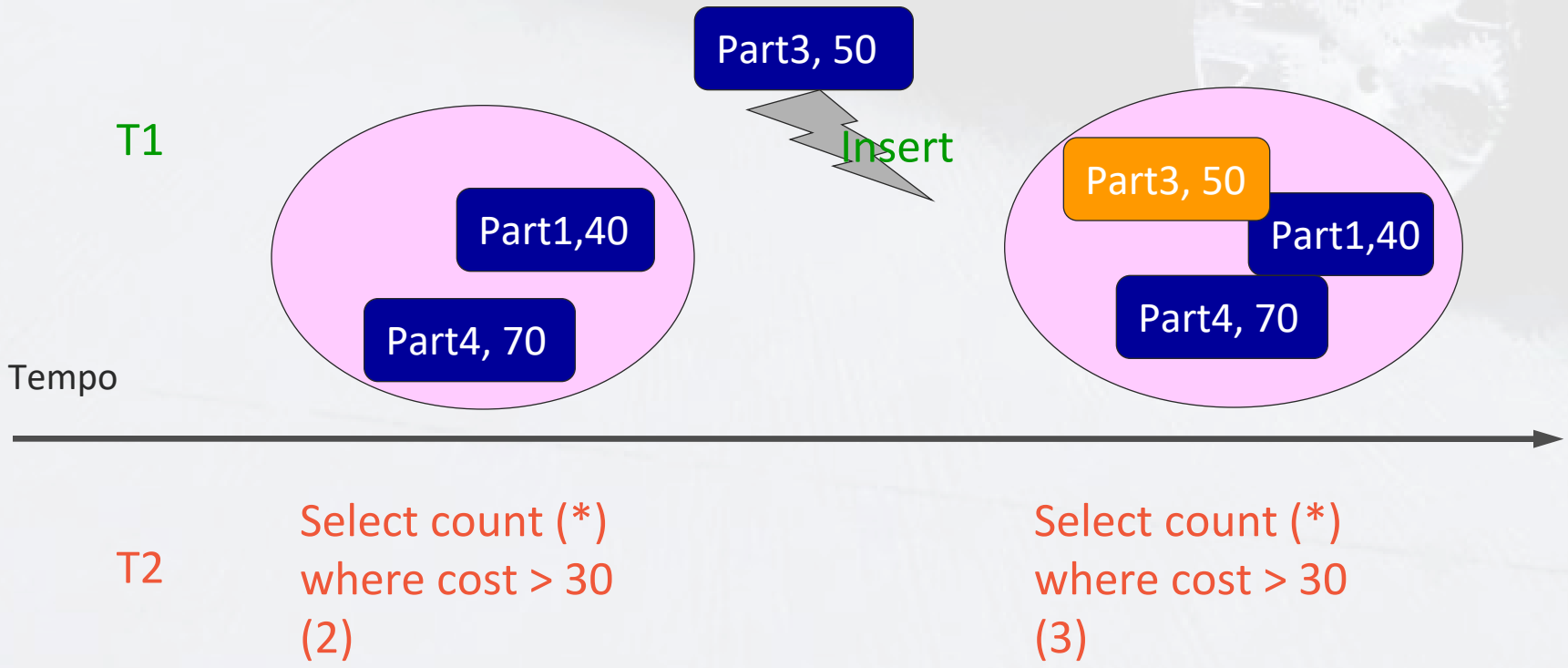
Letture non ripetibili

- Lettura successiva inconsistente



Letture fantasma

- Lettura intervallo successiva inconsistente



Livelli di isolamento in SQL Server 2005+

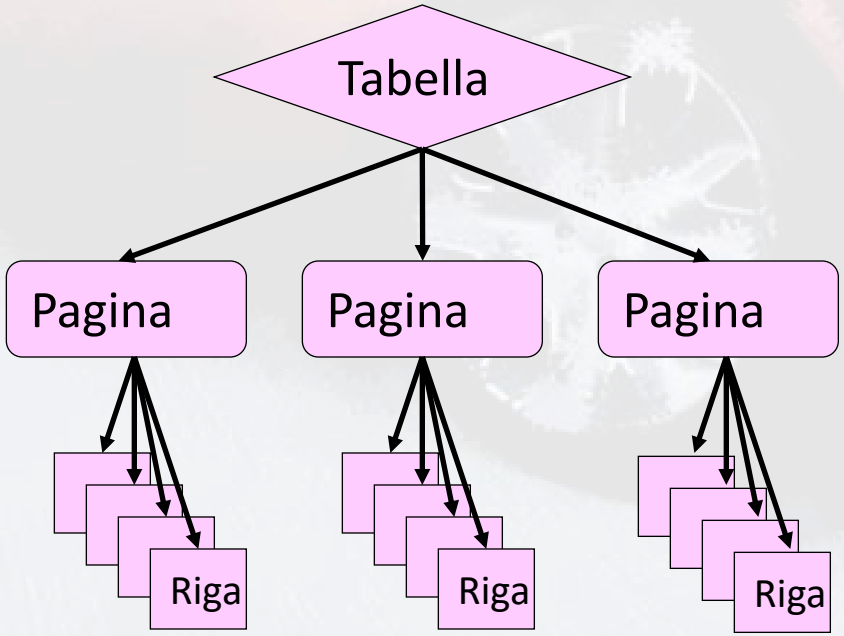
- 4 livelli supportati per standard ANSI
  - Serializzato e 3 livello più deboli che permettono fenomeni precedenti
  - «Read Uncommitted», «Read Committed», «Repeatable Read», «Serializable»
- Livello predefinito «Read Committed» in due modalità
  - pessimistico, usa protocollo di «Locking»
  - ottimistico, usa «Row Versioning»
- Aggiunta di un ulteriore livello «Snapshot»
  - simile a «Serializable» ma ottimistico in scrittura
  - genera errore alla «Commit» in caso di conflitto

Controllo del livello di isolamento

- Opzione a livello di sessione
  - SET TRANSACTION ISOLATION LEVEL
  - API impostano il livello dietro le quinte
- «Table Hints»
  - livelli di isolamento legati al funzionamento del lock manager
  - «aiuti» nelle richieste per forzare il lock manager
  - ignorano l’impostazione corrente

Gestione dei «Lock»

- Tipologie di lock
  - Condivisi (S)
  - Esclusivi (X)
  - Aggiornamento (U)
- Intenzione (I)
- Granularità
  - Riga/chiave
  - Intervallo tra chiavi
  - Pagina
  - Tabella
  - «Extent»
  - Database



Sottoinsieme matrice compatibilità «Lock»

	Lock già acquisito							
Lock richiesto	IS	S	U	IX	X	SchS	SchM	BU
IS	Si	Si	Si	Si	No	Si	No	No
S	Si	Si	Si	No	No	Si	No	No
U	Si	Si	No	No	No	Si	No	No
IX	Si	No	No	Si	No	Si	No	No
X	No	No	No	No	No	Si	No	No
SchS	Si	Si	Si	Si	Si	Si	No	Si
SchM	No	No	No	No	No	No	No	No
BU	No	No	No	No	No	Si	No	Si

«Read Committed» (pessimistico)

```
CREATE TABLE t1 ( c1 int unique, c2 int)
INSERT INTO t1 VALUES (1, 5)
```

Transazione 1

```
BEGIN TRAN

UPDATE t1
SET    c2 = 9
WHERE  c1 = 1

COMMIT TRAN
```

Transaction 2
(Read Committed)

```
BEGIN TRAN

SELECT c2 FROM t1
WHERE  c1 = 1

-- SQL Server risponde 9

COMMIT TRAN
```

Bloccato!

Tempo

«Read Committed Snapshot» (ottimistico)

```
CREATE TABLE t1 ( c1 int unique, c2 int)
INSERT INTO t1 VALUES (1, 5)
```

Transazione 1

```
BEGIN TRAN

UPDATE t1
SET    c2 = 9
WHERE  c1 = 1

COMMIT TRAN
```

Transaction 2
(Read Committed Snapshot)

```
BEGIN TRAN

SELECT c2 FROM t1
WHERE  c1 = 1

  -- SQL Server risponde 5

SELECT c2 FROM t1
WHERE  c1 = 1
  -- SQL Server risponde 9

COMMIT TRAN
```

«Snapshot» (in lettura)

```
CREATE TABLE t1 (c1 int unique, c2 int)
INSERT INTO t1 VALUES (1, 5)
```

Transazione 1

```
BEGIN TRAN
UPDATE t1
SET    c2 = 9
WHERE  c1 = 1


COMMIT TRAN
```

Transazione 2 (Snapshot Isolation)

```
SET TRANSACTION ISOLATION LEVEL
SNAPSHOT
BEGIN TRAN
SELECT c2 FROM t1 WHERE c1 = 1
  -- SQL Server risponde 5
SELECT c2 FROM t1 WHERE c1 = 1
  -- SQL Server risponde 5
COMMIT TRAN
SELECT c2 FROM t1 WHERE c1 = 1
  -- SQL Server risponde 9
```

«Snapshot» (in aggiornamento)

```
CREATE TABLE t1 ( c1 int unique, c2 int)
INSERT INTO t1 VALUES (1,5)
```

Transazione 2

```
BEGIN TRAN

UPDATE t1
SET    c2 = 9
WHERE  c1 = 1

COMMIT TRAN
```

transazione 1 (Snapshot Isolation)

```
SET TRANSACTION ISOLATION LEVEL
SNAPSHOT

BEGIN TRAN

SELECT c2 FROM t1 WHERE c1 = 1

  -- SQL Server risponde 5

UPDATE t1
SET    c2 = 15
WHERE  c1 = 1
```

Bloccato!

Rollback perché conflitto tra aggiornamenti

Tempo

Demo

- Isolamento delle transazioni

Livelli, fenomeni, concorrenza

Livelli di isolamento	Fenomeni permessi: Dirty Read	Non-Repeatable Read	Phantoms	Update Conflict	Modello concorrenza
Read Uncommitted	Si	Si	Si	No	
Read Committed 1 Locking *2 Snapshot*	No No	Si Si	Si Si	No No	**Pessimistico** **Ottimistico**
Repeatable Read	No	No	Si	No	**Pessimistico**
Snapshot	No	No	No	Si	**Ottimistico**
Serializable	No	No	No	No	**Pessimistico**

Scelta livello di isolamento

- Mediazione
  - livello alto: più consistenza, meno prestazioni
  - livello basso: meno consistenza, più prestazioni
- Modello pessimistico o ottimistico?
  - dipende… quasi sempre va bene ottimistico
  - è bene censire gli altri casi
  - Impatto sul **tempdb** («version store»)
- Scelta in base alle necessità applicative!
  - non alle prestazioni!

Altri problemi «Read Committed»

- «Shared Lock» rilasciati durante «scan»
- Righe possono muoversi fisicamente
  - Es. aggiornamento indice «Clustered»
  - Fenomeni problematici
    - Rilettura stessa riga
    - Mancata lettura di una riga

Demo

- Problema «Read Committed»

Effetti collaterali implementazione fisica

- Pilastro RDBMS: separazione modello concettuale/logico e fisico
  - «Set based thinking»…
  - «Lock Manager» garantisce equivalenza operazioni…
  - Qualche eccezione dovuta a implementazione fisica…

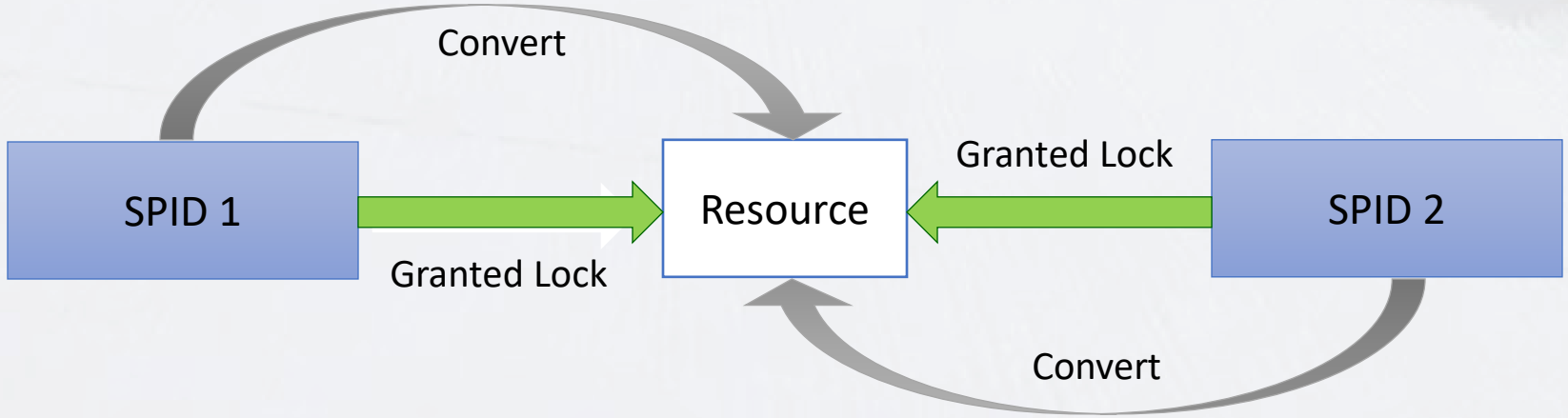
«Deadlock»

«Deadlock» (blocco critico)

- Evento normale
  - tecniche per mitigarne la frequenza
  - non sempre si possono eliminare completamente
- Risoluzione automatica
  - controllo cicli ad intervalli regolari
  - transazione meno costosa per annullamento viene scelta come vittima («rollback»)
  - controllo più frequente quando si verificano «Deadlock»
- Applicazioni
  - ricevono errore 1205
  - devono gestire il problema, es. riprovando

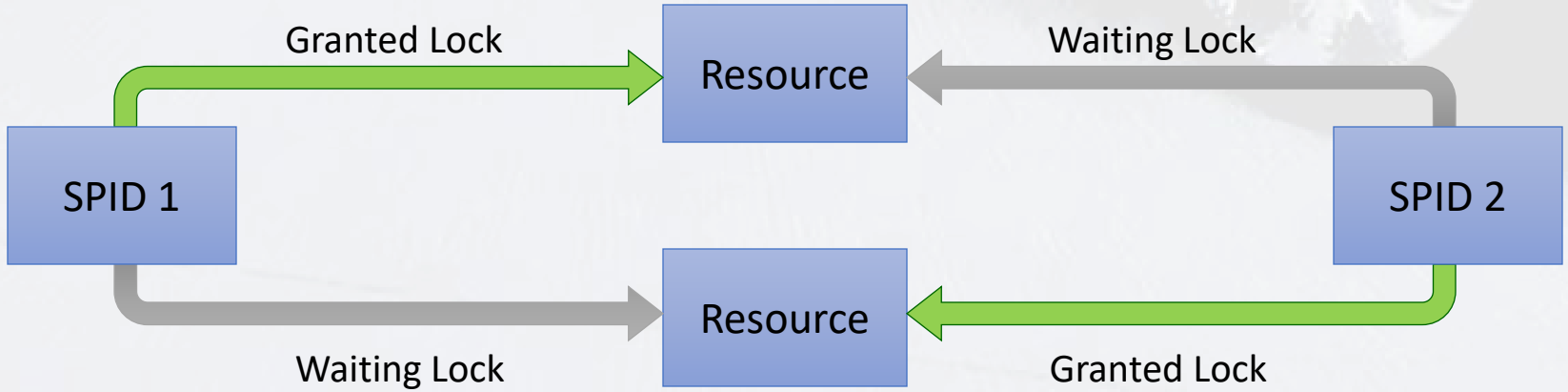
«Conversion Deadlock»

- Coinvolge una risorsa
- Entrambi i processi hanno un «Lock» sulla risorsa
  - tipicamente «Shared»
  - con livello di isolamento più alto di «Read Committed»
- Entrambi cercano di convertire il «Lock»
  - con uno incompatibile



«Cycle Deadlock»

- Coinvolge più risorse
- Processi si bloccano a vicenda in modo ciclico



Demo

- Deadlock

Minimizzare i «Deadlock»

- Usare livello di isolamento appropriato
- Verificare possibilità di usare livelli «Snapshot»
- Controllare gli indici
- Controllare normalizzazione della base dati
- Le transazioni dovrebbero
  - durare poco
  - non richiedere interazione con utente
  - accedere alle risorse nello stesso ordine

Pattern applicativi

Implementazione di «code»

- Selezione PK da scodare e aggiornare lo stato
  - Lock esclusivo mantenuto fino termine transazione
  - Scansioni si bloccano su lock esclusivo, possono saltarlo con «hint» READPAST
  - Prendere esplicitamente lock di aggiornamento con «hint» UPDLOCK
  - Evitare «escalation» con «hint» ROWLOCK

Implementazione di «code» Pattern 1

- BEGIN TRANSACTION
  - SELECT TOP (1) @Id = Id
    FROM QueueTable WITH (**READPAST, UPDLOCK, ROWLOCK**)
    WHERE Status = 0
  - UPDATE QueueTable
    SET Status = 1
    WHERE Id = @id
- COMMIT TRANSACTION

Implementazione di «code» Pattern 2

- UPDATE TOP (1) QueueTable WITH (**READPAST, ROWLOCK**)
  SET Status = 1
  OUTPUT inserted.Id
  WHERE Status = 0

Demo

- Implementazione code

Implementazione di «code» con SQL Broker

- Valutare utilizzo del SQL Server Broker
  - Implementazione nativa code con contratti e messaggi ben definiti
  - Routing multi-istanza
  - «Scodamento» tramite procedure di attivazione (anche .NET)
  - Implementazione singolo database ottimizzata (1 tabella per coda TX/RX)

Lock applicativi

- Stesso modello Lock Manager utilizzabile in T-SQL
  - sp_getapplock
  - sp_releaseapplock
  - APPLOCK_MODE()
  - APPLOCK_TEST()
- Esempio: semafori applicativi

Demo

- Lock applicativi

Risorse

- Libri
  - Expert SQL Server Transactions and Locking: Concurrency Internals for SQL Server Practitioners
    - Dmitri Korotkevitch, ISBN 978-1484239568
  - SQL Server Concurrency: Locking, Blocking, and Row Versioning
    - Kalen Delaney, ASIN B00CZ1BVGY
    - https://www.red-gate.com/library/sql-server-concurrency-locking-blocking-and-row-versioning